

Detailed Documentation of different loading cases in Discrete Dislocation Dynamics Simulation Using OpenDiS

Contents

1	Introduction	3
1.1	installing OpenDis on Ubuntu	3
1.2	Running The code	4
2	Objective of the Simulation	4
3	Code Overview For Uniaxial Stress Controlled Load	4
3.1	Helper Functions	5
3.1.1	Voigt Tensor Conversion	5
3.1.2	Von Mises Stress	6
3.1.3	Equivalent Strain	6
3.1.4	Elastic Constants	6
3.2	Custom Simulation Class	6
3.2.1	Initialization Parameters	7
3.3	Stress-Controlled Constitutive Update	7
3.3.1	Plastic Strain Increment Extraction	7
3.3.2	Stress Increment	7
3.3.3	Elastic Strain Increment	7
3.3.4	Total Strain Update	8
3.4	Stopping Criterion	8
3.5	Output Writing	8
3.6	Simulation Setup	9
3.6.1	Initial Dislocation Structure	9
4	Extension to Uniaxial Strain-Rate Loading	9
4.1	Helper Functions	9
4.2	Custom Simulation Class	10
4.3	Strain-Rate Constitutive Update	10
4.4	Stopping Criterion	12
4.5	Output Writing	12
4.6	Simulation Setup	13

5	Extension to Cyclic Strain-Controlled Loading	13
5.1	Cyclic Strain History	14
5.2	Elastic-Plastic Stress Update	14
5.3	Plastic Strain Rate and Plastic Velocity Gradient	15
5.4	Output Quantities	15
6	ANN CODE	16
6.1	Input Variables and Learning Targets	16
6.2	Neural-Network Architecture	16
6.3	Recursive Rollout Formulation	17
7	LSTM	19

1 Introduction

This report presents a detailed documentation and physical interpretation on how to install OpenDis , running the code for different loading conditions in Discrete Dislocation Dynamics (DDD) simulation implemented. The simulation models the evolution of a FCC dislocation network subjected to different loadings.

The code performs the following major tasks:

- Initialization of a three-dimensional FCC dislocation network by introducing segments using numlines feature in OpenDis.
- Application of a prescribed loading.
- Computation of elastic and plastic strain increments.
- Evolution of dislocation density.
- Calculation of equivalent stress and strain measures.
- Computation of plastic power density.

The generated data can be used for:

- strain hardening analysis,
- constitutive model development,
- machine learning surrogate training,

1.1 installing OpenDis on Ubuntu

- Visit the link-<https://opendis.github.io/OpenDiS/>
- Click on installation → compilation → compile on Linux (ubuntu).
- Paste the two commands in install required packages part in terminal one by one.
- Now go in section How to get the code
- Paste the first command and then paste the three commands in "Download OpenDiS together with submodules" in terminal.
- Once this is done , come back in section compile on Linux and paste "rm -rf build/; ./configure.sh -DSYS=ubuntu" followed by "cmake -build build -j 8 ; cmake -build build -target install" in Build ExaDiS/KOKKOS section.

Before running the code , numpy and matplotlib should be installed and if not install them using command "pip3 install numpy matplotlib".

1.2 Running The code

- Go in the directory where OpenDis is installed, say if it's in Downloads go in Downloads using `cd Downloads`.
- `cd OpenDis → cd core → cd exadis → cd pytho`.
- create a python file using nano command and name the file with .py at end.
- Write the code in the file and once done , click ctrl+o and then enter to save the file.
- Run the file using the command `python3 "filename"`.

2 Objective of the Simulation

The primary objective of the simulation is to generate consistent DDD data under stress-controlled loading conditions.

The applied loading is:

$$\sigma_{zz}(t) = \dot{\epsilon}_{zz}t \quad (1)$$

and

$$\epsilon_{zz}(t) = \dot{\sigma}_{zz}t \quad (2)$$

where $\dot{\sigma}_{zz}$ and $\dot{\epsilon}_{zz}$ are the prescribed stress and strain rate.

The simulation continues until:target stress (in stress controlled loading) or target strain (in strain controlled loading) is reached.

The corresponding loading time is internally computed and this approach ensures that the loading path is directly controlled through loading rate.

3 Code Overview For Uniaxial Stress Controlled Load

The simulation code follows the following sequence:

1. Import and initialization of Python + PyExaDiS/OpenDiS
2. Defining material and numerical parameters like type of crystal , burger vector magnitude , shear modulus , poisson ratio and time step limits
3. Computing elastic constants.
4. Generating initial dislocation network using the function `G = ExaDisNet().generate_line_conf` in our case we are using a box of size 15 microns by introducing segments through `numlines` parameters across all 12 slip systems.
5. Creating DisNetManager which is a class defined in OpenDis manageing the network during the simulation followed by Initialization of DDD modules like `calforce` , `MobilityLaw`, `Timeintegration` , `Collision`, `Remesh` and `Topology`.
6. Next we defined the loading by specifying rate and target value.

7. Time loop was started where at each step we calculated the forces on dislocations, velocities from `MobilityLaw` and updated the node positions handling topology changes and remeshing.
8. plastic strain and plastic spin increment were calculated from `get_plastic_strain()` function in `OpenDiS`.and applied stress was updated.
9. Elastic strain incremented gets computed and total strain is updated in same step”.
10. Ouput quantities like von mises stress, total strain , stress components , dislocation density and equivalent plastic strain are are calculated and written in csv file.
11. The process continues till stopping criteria is reached and then the simulation finally stops.

Table 1: Role of OpenDiS Modules

Module	Purpose
ExaDisNet	Dislocation network generation
DisNetManager	Network management
CalForce	Peach-Koehler force calculation
MobilityLaw	Dislocation mobility evolution
TimeIntegration	Time stepping
Collision	Collision handling
Topology	Topological operations
Remesh	Segment remeshing
SimulateNetwork	Main simulation engine

3.1 Helper Functions

3.1.1 Voigt Tensor Conversion

The helper function:

```
1 def voigt_sym_to_tensor(v6):
```

converts a six-component Voigt vector into a full symmetric tensor.

The Voigt ordering used in the code is:

$$[xx, yy, zz, yz, zx, xy] \quad (3)$$

The corresponding tensor form is:

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{zx} \\ \sigma_{xy} & \sigma_{yy} & \sigma_{yz} \\ \sigma_{zx} & \sigma_{yz} & \sigma_{zz} \end{bmatrix} \quad (4)$$

This conversion is necessary because `OpenDiS` internally stores quantities in Voigt notation while tensor operations require full tensor representation.

3.1.2 Von Mises Stress

The equivalent von Mises stress is computed using:

$$\sigma_{\text{vm}} = \sqrt{\frac{3}{2} \mathbf{s} : \mathbf{s}} \quad (5)$$

where the deviatoric stress tensor is:

$$\mathbf{s} = \boldsymbol{\sigma} - \frac{1}{3} \text{tr}(\boldsymbol{\sigma}) \mathbf{I} \quad (6)$$

3.1.3 Equivalent Strain

The equivalent strain is computed using:

$$\varepsilon_{\text{eq}} = \sqrt{\frac{2}{3} \mathbf{E}' : \mathbf{E}'} \quad (7)$$

where:

$$\mathbf{E}' = \mathbf{E} - \frac{1}{3} \text{tr}(\mathbf{E}) \mathbf{I} \quad (8)$$

This provides a scalar measure of the total strain magnitude.

3.1.4 Elastic Constants

The code computes Young's modulus and Poisson's ratio from Lamé constants.

Young's modulus is calculated as:

$$E = \frac{\mu(3\lambda + 2\mu)}{\lambda + \mu} \quad (9)$$

Poisson's ratio is:

$$\nu = \frac{1}{2} \frac{\lambda}{\lambda + \mu} \quad (10)$$

These parameters are used to construct the isotropic elastic compliance matrix.

3.2 Custom Simulation Class

The class:

```
1 class MySimulateNetworkPerf(SimulateNetwork):
```

extends the default OpenDiS simulation class.

The purpose of this custom class is to override the default OpenDiS simulation class as per our requirement like stress controlled loading , adding stopping criteria based on reaching of target stress without relying on max number os steps, output writing and csv file generation explicitly as per user requirements:

3.2.1 Initialization Parameters

The constructor store parameters like:

- λ = Lamé constant,
- μ = shear modulus,
- `stress_rate.tensor` = applied stress-rate tensor,
- t_{total} = final loading time,
- $\sigma_{zz}^{\text{target}}$ = target axial stress.

The class also initializes output files:

- `stress_strain_dens.dat`
- `stress_strain_dens_main.csv`
- `stress_strain_dens_power.csv`

3.3 Stress-Controlled Constitutive Update

The most important part of the code is:

```
1 step_update_response()
```

This function performs the constitutive update at every time step.

3.3.1 Plastic Strain Increment Extraction

The code obtains:

```
1 dEp_T, dWp_vec, density =  
2 N.get_disnet(ExaDisNet).net.get_plastic_strain()
```

where `dEp_T` is the plastic strain increment and `dWp_T` is the plastic spin increment. The ordering here has been kept constant with what has been mentioned in OpenDis slides.

3.3.2 Stress Increment

The stress increment is applied as:

$$\Delta \sigma = \dot{\sigma} \Delta t \quad (11)$$

For uniaxial loading:

$$\dot{\sigma} = [0, 0, \dot{\sigma}_{zz}, 0, 0, 0] \quad (12)$$

3.3.3 Elastic Strain Increment

The elastic strain increment is computed using isotropic elasticity:

$$\Delta \epsilon^e = \mathbf{S} : \Delta \sigma \quad (13)$$

where $\mathbf{S}$ is the compliance tensor.

3.3.4 Total Strain Update

The total strain increment is:

$$\Delta\epsilon = \Delta\epsilon^e + \Delta\epsilon^p \quad (14)$$

Thus the accumulated total strain becomes:

$$\epsilon_{n+1} = \epsilon_n + \Delta\epsilon^e + \Delta\epsilon^p \quad (15)$$

This forms the constitutive integration framework of the simulation.

3.4 Stopping Criterion

The simulation terminates when either:

$$t \geq t_{\text{ramp}} \quad (16)$$

or:

$$\sigma_{zz} \geq \sigma_{\text{target}} \quad (17)$$

This is implemented through a custom exception:

```
1 class StopSimulation(RuntimeError):
```

which cleanly exits the simulation without causing abrupt termination.

3.5 Output Writing

The function:

```
1 step_write_files()
```

writes the simulation data. Here, .dat file contains two row per step, where second line writes power data

Output Variables The following quantities are exported:

Table 2: Simulation Output Variables

Variable	Physical Meaning
strain_eq	Equivalent total strain
sigma_vm	Von Mises stress
s_ij	Stress tensor components
density	Dislocation density
Lp_ij	Plastic velocity gradient
epdot_eq	Equivalent plastic strain rate
ep_eq	Accumulated plastic strain
p_plast	Plastic power density

3.6 Simulation Setup

The main simulation function is:

```
1 run_stress_controlled()
```

Table 3: Material Parameters

Parameter	Value	Unit
Burgers vector (b)	2.55×10^{-10}	m
Shear modulus (μ)	54.6	GPa
Poisson ratio (ν)	0.324	–

3.6.1 Initial Dislocation Structure

The network is initialized using:

```
1 num_lines = 15*12
2 theta = [0.0, 60.0, 90.0]
```

This generates:

- 180 dislocation lines, which can be changes by changing the value.
- distributed across FCC slip systems,
- with screw, mixed, and edge character.

4 Extension to Uniaxial Strain-Rate Loading

A uniaxial strain-rate controlled DDD simulation was implemented in which the total strain along a prescribed loading direction is imposed at a fixed rate, and the stress evolves as a result of the elastic-plastic constitutive update. The loading mode is set as `loading_mode='strain_rate'` which directly activates the custom update in the driver class.

4.1 Helper Functions

Two helper functions are defined in this code that were not present in the stress-controlled case.

Tensor to Voigt Conversion

```
def tensor_to_voigt_sym(T):
```

This is the inverse of `voigt_sym_to_tensor` used earlier. It converts a 3×3 symmetric tensor back into a six-component Voigt vector. The Voigt ordering is:

$$[xx, yy, zz, yz, zx, xy] \quad (18)$$

This function is needed because the imposed strain increment is first built as a 3×3 tensor and then is converted to Voigt form before the constitutive update.

Isotropic Stiffness Matrix

`def build_C_voigt(LA, MU):`

This function builds the isotropic elastic stiffness matrix **C** in

This stiffness matrix is used to compute the stress increment from the elastic strain increment as:

$$\Delta\boldsymbol{\sigma} = \mathbf{C} : \Delta\boldsymbol{\epsilon}^e \quad (19)$$

This is the main difference from the stress-controlled case. In stress-controlled loading the compliance matrix **S** (inverse of **C**) was used because stress was the prescribed quantity.

-

4.2 Custom Simulation Class

The class:

`class MyStrainRateSimulateNetwork(SimulateNetwork):`

extends the default OpenDiS simulation driver, identical in structure to the stress-controlled case. The constructor stores the following parameters beyond the base class:

- λ, μ — Lamé constant and shear modulus
- `erate` — imposed scalar strain rate (s^{-1})
- `edir` — loading direction vector (3-component). This is normalized to a unit vector **n** inside the constructor
- `max_strain` — target imposed scalar strain at which the simulation stops
- `tol_strain` — tolerance on strain for the stopping criterion

The constructor also initializes the following internal accumulators:

- `_time_cum` — accumulated simulation time
- `_eps_imposed` — accumulated imposed scalar strain
- `_ep_eq_cum` — accumulated equivalent plastic strain

Only one output file is initialized named `stress_strain_dens_main.csv`

4.3 Strain-Rate Constitutive Update

The core update is performed in:

`def step_update_response(self, N: DisNetManager, state: dict):`

This function is called at every simulation step and performs the full elastic-plastic constitutive update. The steps are as follows.

Step 1: Extract Plastic Increment from DDD Network

The plastic strain increment and plastic spin are obtained from the library similar to the stress-controlled case:

```
dEp_T, dWp_vec, density =  
    N.get_disnet(ExaDisNet).net.get_plastic_strain()
```

The plastic strain increment is reordered from the raw ExaDiS tensor output to Voigt convention $[xx, yy, zz, yz, xz, xy]$ and the plastic spin is stored as $[yz, xz, xy]$.

Step 2: Clamping the Time Step

The raw OpenDiS adaptive time step dt is clamped so that the accumulated imposed strain lands exactly on `max_strain` and does not overshoot:

$$dt_{\text{eff}} = \min\left(dt, \frac{\varepsilon_{\text{max}} - \varepsilon_{\text{imposed}}}{\dot{\varepsilon}}\right) \quad (20)$$

where $\varepsilon_{\text{imposed}}$ is the strain accumulated and $\dot{\varepsilon}$ is the prescribed strain rate. After clamping, the accumulated time and imposed strain are updated:

Step 3: Elastic Strain Increment

The elastic strain increment is computed by subtracting the plastic strain increment (from the DDD network) from the total imposed strain increment:

$$\Delta\boldsymbol{\varepsilon}^e = \Delta\boldsymbol{\varepsilon} - \Delta\boldsymbol{\varepsilon}^p \quad (21)$$

Step 4: Stress Increment

The stress increment is computed using the isotropic stiffness matrix $\mathbf{C}$ built in `build_C_voigt`:

$$\Delta\boldsymbol{\sigma} = \mathbf{C} : \Delta\boldsymbol{\varepsilon}^e \quad (22)$$

In Voigt notation this is simply a matrix-vector product:

```
dS_voigt = self.C @ dEe
```

Step 5: Accumulate Total Strain and Stress

The total strain and applied stress are updated recursively:

$$\boldsymbol{\varepsilon}^{n+1} = \boldsymbol{\varepsilon}^n + \Delta\boldsymbol{\varepsilon} \quad (23)$$

$$\boldsymbol{\sigma}^{n+1} = \boldsymbol{\sigma}^n + \Delta\boldsymbol{\sigma} \quad (24)$$

The von Mises stress and equivalent strain scalars are then computed from these accumulated tensors in the same way as the stress-controlled case.

4.4 Stopping Criterion

The stopping criterion is checked inside:

```
def step_end(self, N: DisNetManager, state: dict):
```

The simulation terminates when the accumulated imposed scalar strain reaches the target:

$$\varepsilon_{\text{imposed}} \geq \varepsilon_{\text{max}} - \varepsilon_{\text{tol}} \quad (25)$$

4.5 Output Writing

The function:

```
def step_write_files(self, N: DisNetManager, state: dict):
```

writes one row per step to the CSV file, controlled by `write_freq`. Setting `write_freq=1` writes every single step.

The plastic velocity gradient $\mathbf{L}^p$ and equivalent plastic strain rate are computed identically to the stress-controlled case:

$$\mathbf{L}^p = \frac{\Delta \boldsymbol{\varepsilon}^p + \Delta \mathbf{W}^p}{dt_{\text{eff}}} \quad (26)$$

$$\dot{\varepsilon}_{\text{eq}}^p = \sqrt{\frac{2}{3} \mathbf{D}^p : \mathbf{D}^p}, \quad \mathbf{D}^p = \frac{\Delta \boldsymbol{\varepsilon}^p}{dt_{\text{eff}}} \quad (27)$$

$$\varepsilon_{\text{eq}}^{p,n+1} = \varepsilon_{\text{eq}}^{p,n} + \dot{\varepsilon}_{\text{eq}}^p dt_{\text{eff}} \quad (28)$$

An additional function:

```
def step_print_info(self, N: DisNetManager, state: dict):
```

is overridden to print step number, time, `dt_eff`, `eps_imposed`, `strain_eq`, and `sigma_vm` to the terminal at intervals controlled by `print_freq`.

Table 4: Output variables for strain-rate controlled DDD simulation

Variable	Physical Meaning
step	Simulation step number
time(s)	Accumulated simulation time
dt(s)	Raw OpenDiS adaptive time step
dt_eff(s)	Clamped effective time step used for update
eps_imposed	Accumulated imposed scalar strain
strain_eq	Equivalent total strain (von Mises measure)
sigma_vm(Pa)	Von Mises stress
s_xx, s_yy, s_zz	Normal stress components
s_yz, s_zx, s_xy	Shear stress components
density(1/m ²)	Dislocation density
Lp_ij	Plastic velocity gradient components (9 values)
epdot_eq(1/s)	Equivalent plastic strain rate
ep_eq(—)	Accumulated equivalent plastic strain

4.6 Simulation Setup

The main function:

```
def run_strain_controlled():
```

sets up and runs the full simulation. The material parameters are identical to the stress-controlled case and are listed in Table 3. The initial dislocation microstructure is also identical: a $15\text{ }\mu\text{m}$ FCC box with dislocation lines distributed across all 12 slip systems with screw, mixed, and edge character (`theta = [0.0, 60.0, 90.0]`).

The changeable loading parameters are:

Table 5: User-changeable loading parameters for strain-rate controlled simulation

Parameter	Default Value	How to Change
<code>erate</code>	$1 \times 10^4\text{ s}^{-1}$	Change for different strain rates
<code>edir</code>	[0, 0, 1]	Change for different loading directions
<code>max_strain</code>	0.005	Change for different target strains
<code>write_dir</code>	"straincontrol1"	Change for every new simulation

5 Extension to Cyclic Strain-Controlled Loading

In addition to monotonic stress-controlled loading, a cyclic strain-controlled DDD simulation was also implemented. The objective is to study the response of the dislocation network under repeated forward and reverse deformation. OpenDis has no built-in cyclic strain control. So the entire loading mechanism had to be built manually by overriding the simulation driver. Here, we directly prescribe the total strain history along a selected loading direction. The loading mode='stress' workaround in the cyclic driver is used to satisfy ExaDiS initialization while overriding the actual update

The cyclic loading is imposed using a custom driver class,

```
1 class CyclicStrainDriver(SimulateNetwork):
```

which extends the OpenDiS simulation driver and overrides the macroscopic update procedure. One complete loading cycle is defined as

$$0 \rightarrow +\varepsilon_{\max} \rightarrow -\varepsilon_{\max} \rightarrow 0. \quad (29)$$

For the present simulation, the strain amplitude is prescribed as

$$\varepsilon_{\max} = 0.0035, \quad (30)$$

which corresponds to 0.35% strain. The applied strain rate is

$$\dot{\varepsilon} = 5.0 \times 10^3\text{ s}^{-1}. \quad (31)$$

The loading is applied along the y -direction, so the imposed strain increment is

$$\Delta\boldsymbol{\varepsilon} = [0, \Delta\varepsilon_{yy}, 0, 0, 0, 0]. \quad (32)$$

5.1 Cyclic Strain History

The cyclic strain history is generated using a piecewise linear function. The duration of one loading leg is

$$t_{\text{leg}} = \frac{\varepsilon_{\text{max}}}{\dot{\varepsilon}}. \quad (33)$$

Since one complete cycle consists of four strain legs, the total time for one cycle is

$$t_{\text{cycle}} = 4t_{\text{leg}}. \quad (34)$$

For N_{cycle} cycles, the total simulation time becomes

$$t_{\text{total}} = N_{\text{cycle}}t_{\text{cycle}}. \quad (35)$$

The imposed cyclic strain is therefore controlled by the function

$$\varepsilon_{yy}(t) = \begin{cases} \dot{\varepsilon}t, & 0 \leq t < t_{\text{leg}}, \\ \varepsilon_{\text{max}} - \dot{\varepsilon}(t - t_{\text{leg}}), & t_{\text{leg}} \leq t < 3t_{\text{leg}}, \\ -\varepsilon_{\text{max}} + \dot{\varepsilon}(t - 3t_{\text{leg}}), & 3t_{\text{leg}} \leq t < 4t_{\text{leg}}. \end{cases} \quad (36)$$

This function is repeated for the prescribed number of cycles.

5.2 Elastic-Plastic Stress Update

At every time step, the code first computes the imposed strain increment from the difference between the new and old target strain values:

$$\Delta\varepsilon_{yy} = \varepsilon_{yy}^{n+1} - \varepsilon_{yy}^n. \quad (37)$$

The total imposed strain increment is then written in Voigt notation as

$$\Delta\boldsymbol{\varepsilon} = [0, \Delta\varepsilon_{yy}, 0, 0, 0, 0]. \quad (38)$$

The plastic strain increment is obtained directly from the evolving DDD network:

$$\Delta\boldsymbol{\varepsilon}^p = \text{DDD network output}. \quad (39)$$

The elastic strain increment is calculated as

$$\Delta\boldsymbol{\varepsilon}^e = \Delta\boldsymbol{\varepsilon} - \Delta\boldsymbol{\varepsilon}^p. \quad (40)$$

The stress increment is then obtained using isotropic linear elasticity:

$$\Delta\boldsymbol{\sigma} = \lambda \text{tr}(\Delta\boldsymbol{\varepsilon}^e) \mathbf{I} + 2\mu\Delta\boldsymbol{\varepsilon}^e. \quad (41)$$

In Voigt notation, this is implemented as

$$\Delta\boldsymbol{\sigma} = \lambda \sum_{i=1}^3 \Delta\varepsilon_{ii}^e [1, 1, 1, 0, 0, 0] + 2\mu\Delta\boldsymbol{\varepsilon}^e. \quad (42)$$

The applied stress is updated recursively as

$$\boldsymbol{\sigma}^{n+1} = \boldsymbol{\sigma}^n + \Delta\boldsymbol{\sigma}. \quad (43)$$

Similarly, the total strain is updated as

$$\boldsymbol{\epsilon}^{n+1} = \boldsymbol{\epsilon}^n + \Delta\boldsymbol{\epsilon}. \quad (44)$$

5.3 Plastic Strain Rate and Plastic Velocity Gradient

The symmetric plastic strain-rate tensor is computed from the plastic strain increment as

$$\mathbf{D}^p = \frac{\Delta\boldsymbol{\epsilon}^p}{\Delta t}. \quad (45)$$

The plastic spin increment $\Delta\mathbf{W}^p$ is also obtained from the DDD network. Therefore, the full plastic velocity gradient is computed as

$$\mathbf{L}^p = \frac{\Delta\boldsymbol{\epsilon}^p + \Delta\mathbf{W}^p}{\Delta t}. \quad (46)$$

The equivalent plastic strain rate is evaluated as

$$\dot{\epsilon}_{\text{eq}}^p = \sqrt{\frac{2}{3} \mathbf{D}^p : \mathbf{D}^p}. \quad (47)$$

The accumulated equivalent plastic strain is updated by time integration:

$$\epsilon_{\text{eq}}^{p,n+1} = \epsilon_{\text{eq}}^{p,n} + \dot{\epsilon}_{\text{eq}}^p \Delta t. \quad (48)$$

5.4 Output Quantities

The cyclic simulation writes the output file

```
1 cyclic_strain_control.csv
```

The main output quantities are:

Table 6: Output variables for cyclic strain-controlled DDD simulation

Variable	Physical meaning
time_user(s)	accumulated simulation time
dt_eff(s)	effective time increment
cycle_index	current loading cycle number
cycle_time(s)	local time within the current cycle
strain_eq	equivalent total strain
sigma_vm(Pa)	von Mises stress
eps_target(-)	imposed cyclic strain value
s_xx, s_yy, s_zz	normal stress components
s_yz, s_xz, s_xy	shear stress components
Etot_yy	imposed total strain in loading direction
density(1/m ²)	dislocation density
Lp_ij	plastic velocity gradient components
epdot_eq(1/s)	equivalent plastic strain rate
ep_eq(-)	accumulated equivalent plastic strain

6 ANN CODE

We replaced repeated discrete dislocation dynamics (DDD) simulations by a fast constitutive model that predicts (i) the plastic deformation rate and (ii) the dislocation-density evolution at each time step. The implementation in our work uses two separate neural-network branches: a plastic-flow network and a density-evolution network. This design was adopted to improve the stability over long loading paths. This also ensures stable gradients while training treating plastic flow and density evolution separately.

6.1 Input Variables and Learning Targets

The plastic-flow network receives a 10-dimensional input vector at each step:

$$\mathbf{x}_{Dp}^{(k)} = [\sigma_{xx}^{(k)}, \sigma_{yy}^{(k)}, \sigma_{zz}^{(k)}, \sigma_{yz}^{(k)}, \sigma_{zx}^{(k)}, \sigma_{xy}^{(k)}, \dot{\sigma}, \log_{10} \rho^{(k)}, m, \varepsilon_{eq}^{p,(k)}]^T \quad (49)$$

where, $\dot{\sigma}$ is the prescribed loading rate for the run and $m \in \{0, 1\}$ is the loading-mode indicator ($m = 0$ for uniaxial loading and $m = 1$ for shear loading). The density-evolution network receives the same state 10 variables together with the local time step Δt .

Here, Each input variable has a clear physical significance. The stress components define the driving force for dislocation motion. The prescribed stress rate captures loading-rate effects. The logarithmic dislocation density represents the internal microstructural state. The accumulated plastic strain accounts for deformation history, affecting material evolution. Finally, the loading mode indicator distinguishes between uniaxial and shear loading conditions, enabling the model to learn mode-dependent responses. All inputs are normalized to a common scale before being passed into the neural network and all inputs are normalized to a common scale before being passed into the neural network ensuring numerical stability and efficient learning. This is important as the variables span multiple orders of magnitude, especially dislocation density and normalization prevents dominance of large-scale variables in the loss function.

The constitutive surrogate consists of two mappings defined at each time step k .

The plastic network is expressed as

$$\mathbf{D}_{(k)}^p = \mathcal{F}_1 \left(\boldsymbol{\sigma}_{(k)}, \dot{\sigma}, \rho_{(k)}, m, \varepsilon_{eq,(k)}^p \right) \quad (50)$$

The evolution of dislocation density is given by

$$\frac{d}{dt} \log \rho_{(k)} = \mathcal{F}_2 \left(\boldsymbol{\sigma}_{(k)}, \dot{\sigma}, \rho_{(k)}, m, \varepsilon_{eq,(k)}^p \right) \quad (51)$$

where $\boldsymbol{\sigma}_{(k)}$ is the stress tensor.

6.2 Neural-Network Architecture

Two separate neural networks for plastic flow and density evolution is used because of the distinct physical nature of these processes. Plastic strain rate depends primarily on the instantaneous stress state, whereas dislocation density evolution is strongly history-dependent and keeps evolving. A single network was found to be insufficient to accurately capture both behaviors simultaneously, and separating them improves stability and predictive accuracy.

Plastic-flow network The model takes a 10-dimensional input and gives 5 outputs using the layer sequence

$$10 \rightarrow 128 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 5$$

Each hidden layer uses the SiLU activation function and dropout with probability 0.05.

Density-evolution network The model takes an 11-dimensional input and gives a scalar output using the architecture

$$11 \rightarrow 128 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 1.$$

Again, SiLU activations and dropout of 0.05 are used. All linear layers are initialized with zero bias initialization.

6.3 Recursive Rollout Formulation

After training, the surrogate is tested in recursive rollout mode. At any step k , the plastic flow branch predicts the components of plastic rate tensor $\mathbf{D}^p$,

from which the equivalent plastic strain rate is computed and integrated as

$$\varepsilon_{\text{eq}}^{p,(k+1)} = \varepsilon_{\text{eq}}^{p,(k)} + \dot{\varepsilon}_{\text{eq}}^{p,(k)} \Delta t_k \quad (52)$$

The density branch predicts the logarithmic density rate and advances the density as

$$\log_{10} \rho^{(k+1)} = \log_{10} \rho^{(k)} + \left(\frac{d \log_{10} \rho}{dt} \right)^{(k)} \Delta t_k \quad (53)$$

which finally gives $\rho^{(k)} = 10^{\log_{10} \rho^{(k)}}$ Two rollout modes are used for analysis.

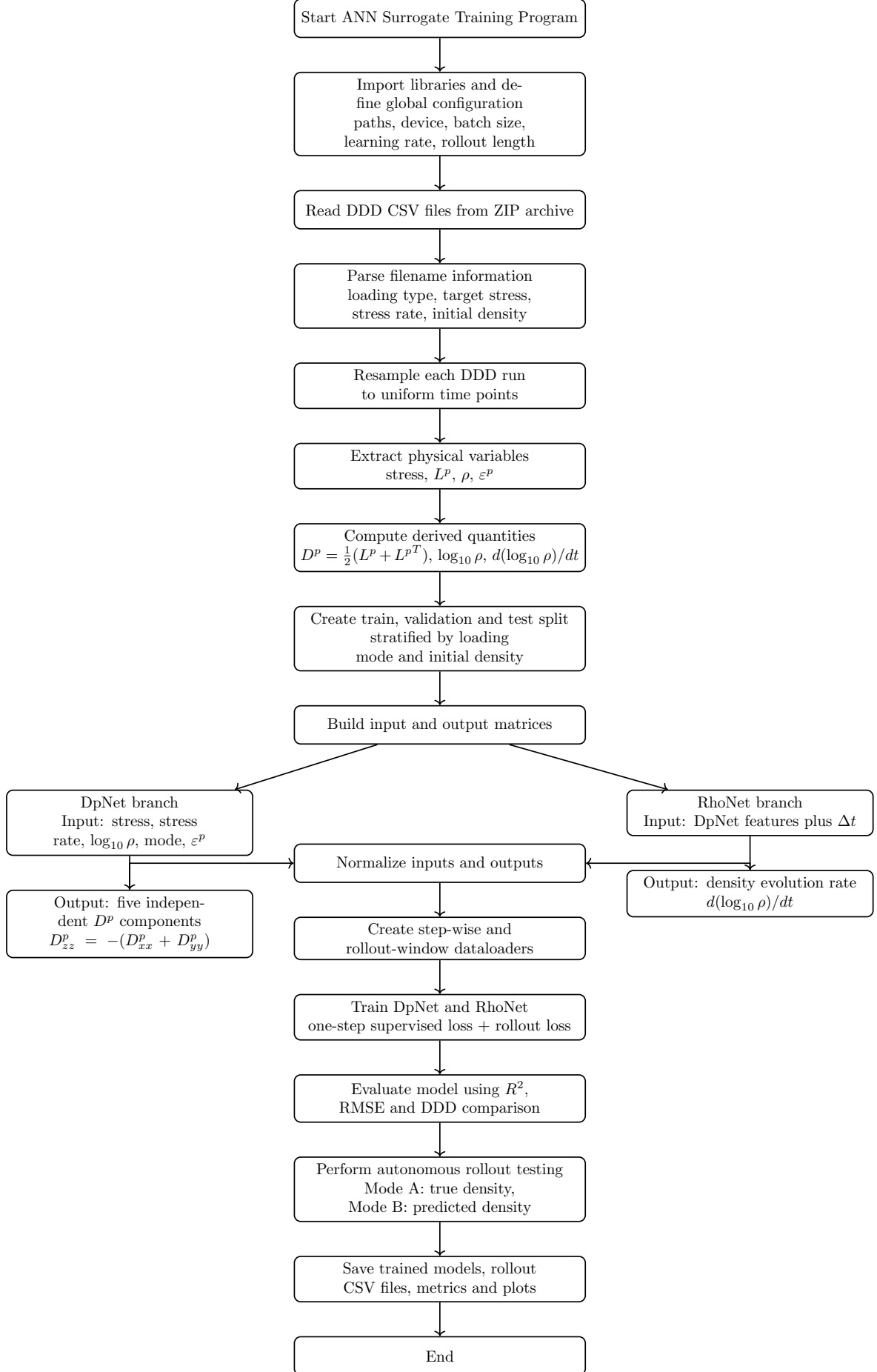


Figure 1: Flowchart of the ANN-based constitutive surrogate training framework using DDD data.

7 LSTM

In addition to the feed-forward neural-network surrogate, a recurrent LSTM-based surrogate was also trained and tested on the same DDD database. The purpose of this comparison was to examine whether providing a sequence history improves the prediction of plastic flow and dislocation-density evolution.

The LSTM surrogate was also divided into two branches. The first branch predicts the five independent components of the plastic strain-rate tensor, while the second branch predicts the logarithmic dislocation-density evolution rate. The input sequence length used for the LSTM model was $H = 24$. The same grouped run-level split was used, consisting of 52 training runs, 8 validation runs, and 8 test runs.

The plastic-flow LSTM receives the input sequence

$$\mathbf{x}_{Dp}^{(k)} = [\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{yz}, \sigma_{zx}, \sigma_{xy}, \dot{\sigma}, \log_{10} \rho, m, \varepsilon_{eq}^p]^{(k)}$$

and predicts

$$[D_{xx}^p, D_{yy}^p, D_{yz}^p, D_{xz}^p, D_{xy}^p]^{(k)}$$

The remaining normal component was reconstructed using plastic incompressibility,

$$D_{zz}^p = -(D_{xx}^p + D_{yy}^p)$$

The density LSTM uses the same input variables with the additional time-step value Δt , and predicts the logarithmic density rate. During recursive rollout, the predicted equivalent plastic strain and predicted density are fed back into the model,

The LSTM model uses the mapping

$$\mathcal{F}_\theta : \left(\boldsymbol{\sigma}_{(k)}, \dot{\sigma}, \rho_{(k)}, m, \varepsilon_{eq,(k)}^p \right) \rightarrow (\dot{\varepsilon}^p, \dot{\rho})$$

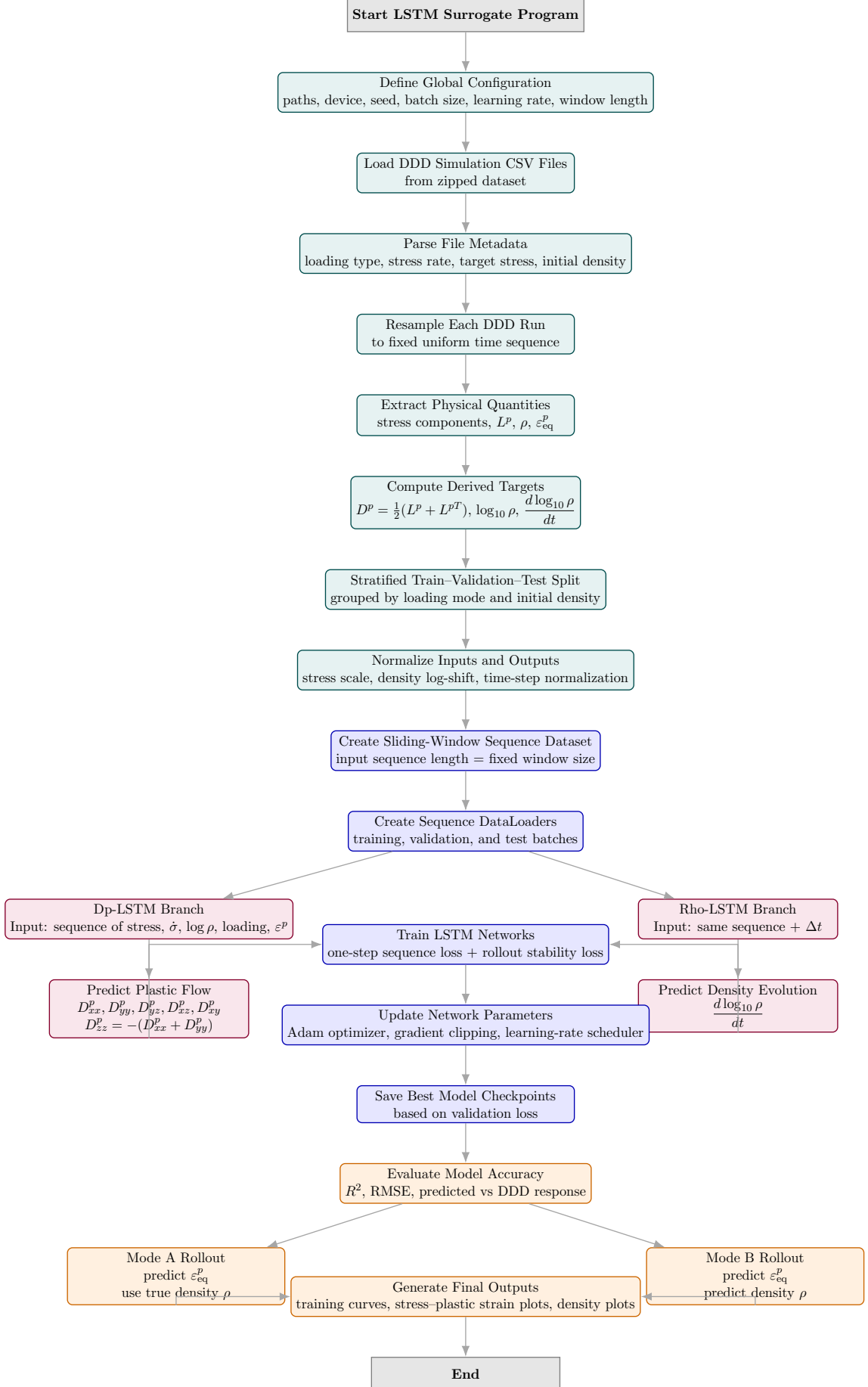


Figure 2: Flowchart of the LSTM-based surrogate modelling pipeline for DDD data